UCSM Esc. Prof. de Ingeniería de Sistemas

Marzo - 2026

Informe de Prácticas

ALGORITMOS Y ESTRUCTURA DE DATOS

Práctica N° 06: POO: Clases y Arreglos de Objetos

Elaborado por:
LUCA DANIEL ESPINOZA RODRÍGUEZ
LUIS SANTIAGO HERMOZA RAMOS
JHOEL ITALO INCAHUANACO YUCRA

© IEEE 2013 The Institute of Electrical and Electronics Engineers, Inc.

GRUPO N°06
ALGORITMOS Y ESTRUCTURA DE DATOS

Código	Nombre	Desarrollo
2024003125	Espinoza Rodríguez, Luca Daniel	100%
2024003229	Hermoza Ramos, Luis Santiago	100%
2024004128	Incahuanaco Yucra, Jhoel Italo	100%

Índice de Contenidos

1 Introducción **4**

2 Objetivos **4**

3 Actividades **5**
3.1 Actividad N°1: Creando Rectángulos 5

4 Ejercicios **10**
4.1 Ejercicio N°1: Contenedor de Rectángulos 10
4.2 Ejercicio N°2: Análisis de una Zona Minera 10

5 Dificultades Encontradas **15**

6 Soluciones Implementadas **15**

7 Conclusiones **16**

8 Bibliografía **16**

9 Anexos **16**

1 Introducción

En Algoritmos y Estructuras de Datos, una de las primeras cosas que aprendes es a organizar la información para que sea fácil de trabajar con ella. La Programación Orientada a Objetos viene muy bien, porque te permite juntar los datos y lo que puedes hacer con ellos en una sola cosa llamada clase. Así el código queda más ordenado y es más fácil de entender.
En este laboratorio usamos Java para crear estructuras de datos con clases y arreglos de objetos. Trabajamos con figuras geométricas como coordenadas y rectángulos, e hicimos una clase que sirve como una especie de "caja" donde puedes guardar varios rectángulos junto con su información calculada. También implementamos algoritmos para saber cómo se relacionan dos rectángulos en el espacio y para calcular qué tanto se superponen, lo que muestra bastante bien cómo los algoritmos y las estructuras de datos se complementan para resolver problemas concretos.

2 Objetivos

El objetivo del laboratorio fue implementar clases y objetos en Java usando los conceptos básicos de Programación Orientada a Objetos, como encapsulamiento, constructores y métodos, para resolver problemas de geometría y manejo de datos. Para esto, creamos clases como Coordenada, Rectangulo y Verificador, que nos permitieron representar figuras geométricas y analizar cómo se relacionan entre sí en el espacio. También usamos arreglos de objetos para organizar la información dentro de una clase contenedora llamada ContainerRect.

Por último, leímos datos desde un archivo .txt para cargar una matriz de objetos y encontrar la subregión más valiosa recorriendo esa matriz con bucles anidados.

3 Actividades

3.1 Actividad N°1: Creando Rectángulos

1. Cree un proyecto en Java llamado PyPooAct1, en el que crea las clases del diagrama, las implemente y complete lo que falte de acuerdo a las indicaciones.

```java
import java.util.*;

public class Rectangulo {
    private Coordenada esquina1;
    private Coordenada esquina2;

    public Rectangulo(Coordenada c1, Coordenada c2) {
        double minX, minY, maxX, maxY;

        if (c1.getX() < c2.getX()) {
            minX = c1.getX();
            maxX = c2.getX();
        } else {
            minX = c2.getX();
            maxX = c1.getX();
        }

        if (c1.getY() < c2.getY()) {
            minY = c1.getY();
            maxY = c2.getY();
        } else {
            minY = c2.getY();
            maxY = c1.getY();
        }

        this.esquina1 = new Coordenada(minX, minY);
        this.esquina2 = new Coordenada(maxX, maxY);
    }

    public void setEsquina1(Coordenada coo) {
        this.esquina1 = new Coordenada(coo);
    }

    public void setEsquina2(Coordenada coo) {
        this.esquina2 = new Coordenada(coo);
    }

    public Coordenada getEsquina1() {
        return esquina1;
    }

    public Coordenada getEsquina2() {
        return esquina2;
    }

    public double calculoArea() {
        double base = esquina2.getX() - esquina1.getX();
        double altura = esquina2.getY() - esquina1.getY();
        return base * altura;
    }

    public String toString() {
        return "(" + esquina1 + ", " + esquina2 + ")";
    }
}
```

```java
import java.util.*;

public class Coordenada {
    private double x;
```

```java
    private double y;

    public Coordenada() {
        this.x = 0;
        this.y = 0;
    }

    public Coordenada(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public Coordenada(Coordenada c) {
        this.x = c.getX();
        this.y = c.getY();
    }

    void setX(double x) {
        this.x = x;
    }

    void setY(double y) {
        this.y = y;
    }

    double getX() {
        return x;
    }

    double getY() {
        return y;
    }

    double distancia(Coordenada c) {
        double dif_x = this.x - c.getX();
        double dif_y = this.y - c.getY();
        return Math.sqrt(dif_x * dif_x + dif_y * dif_y);
    }

    static double distancia(Coordenada c1, Coordenada c2) {
        double dif_x = c1.getX() - c2.getX();
        double dif_y = c1.getY() - c2.getY();
        return Math.sqrt(dif_x * dif_x + dif_y * dif_y);
    }

    public String toString() {
        return "[" + x + ", " + y + "]";
    }
}
```

2. Declare e implemente la clase Principal que contenga el método main que realice lo siguiente:

   a. Crear dos objetos Rectángulo (A y B) cuyos lados son paralelos a los ejes X y Y, y que son valores ingresados por el usuario (debe asumir que el usuario ingresa dos esquinas distintas para el rectángulo. Debe asumir que el rectángulo tiene siempre un área positiva. NO necesita comprobar éstas condiciones).

   La función `esSobrePos` revisa si los rectángulos están separados en algún eje, y si no lo están en ninguno, entonces se superponen. `esJunto` descarta primero la sobreposición y luego revisa si comparten exactamente un borde. `esDisjunto` simplemente es la negación de los dos anteriores.

```java
public class Verificador {
    public static boolean esSobrePos(Rectangulo r1, Rectangulo r2) {
        double r1_minX = r1.getEsquina1().getX();
        double r1_maxX = r1.getEsquina2().getX();
        double r1_minY = r1.getEsquina1().getY();
        double r1_maxY = r1.getEsquina2().getY();

        double r2_minX = r2.getEsquina1().getX();
        double r2_maxX = r2.getEsquina2().getX();
        double r2_minY = r2.getEsquina1().getY();
        double r2_maxY = r2.getEsquina2().getY();

        boolean separadoHrz = (r1_maxX <= r2_minX) || (r1_minX >= r2_maxX);
        boolean separadoVrc = (r1_maxY <= r2_minY) || (r1_minY >= r2_maxY);

        if (!separadoHrz && !separadoVrc) {
            return true;
        } else {
            return false;
        }
    }

    public static boolean esJunto(Rectangulo r1, Rectangulo r2) {
        if (esSobrePos(r1, r2)) {
            return false;
        }

        double r1_minX = r1.getEsquina1().getX();
        double r1_maxX = r1.getEsquina2().getX();
        double r1_minY = r1.getEsquina1().getY();
        double r1_maxY = r1.getEsquina2().getY();

        double r2_minX = r2.getEsquina1().getX();
        double r2_maxX = r2.getEsquina2().getX();
        double r2_minY = r2.getEsquina1().getY();
        double r2_maxY = r2.getEsquina2().getY();

        double mayMinX = (r1_minX > r2_minX) ? r1_minX : r2_minX;
        double menMaxX = (r1_maxX < r2_maxX) ? r1_maxX : r2_maxX;
        boolean comparteX = mayMinX < menMaxX;

        double mayMinY = (r1_minY > r2_minY) ? r1_minY : r2_minY;
        double menMaxY = (r1_maxY < r2_maxY) ? r1_maxY : r2_maxY;
        boolean comparteY = mayMinY < menMaxY;

        boolean juntoX = (r1_maxX == r2_minX) || (r1_minX == r2_maxX);
        boolean juntoY = (r1_maxY == r2_minY) || (r1_minY == r2_maxY);

        if ((juntoX && comparteY) || (juntoY && comparteX) || (juntoX && juntoY)) {
            return true;
        } else {
            return false;
        }
    }

    public static boolean esDisjunto(Rectangulo r1, Rectangulo r2) {
```

```
        if (!esSobrePos(r1, r2) && !esJunto(r1, r2)) {
            return true;
        } else {
            return false;
        }
    }
}
```

El main pide al usuario las esquinas de dos rectángulos, los crea y luego usa `Verificador` para saber si se superponen, se juntan o son disjuntos. Si se superponen, llama a `rectanguloS` que calcula el rectángulo de intersección y muestra su área.

```
import java.util.Scanner;

public class Main {

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Ingrese una esquina del 1er rectangulo: ");
    double x1 = sc.nextDouble();
    double y1 = sc.nextDouble();
    Coordenada a1 = new Coordenada(x1, y1);

    System.out.print("Ingrese la esquina opuesta del 1er rectangulo: ");
    double x2 = sc.nextDouble();
    double y2 = sc.nextDouble();
    Coordenada a2 = new Coordenada(x2, y2);

    Rectangulo A = new Rectangulo(a1, a2);

    System.out.print("Ingrese una esquina del 2do rectangulo: ");

    ...


    System.out.println("\nRectangulo A = " + A);
    System.out.println("Rectangulo B = " + B);

    if (Verificador.esSobrePos(A, B)) {
      System.out.println("Rectangulos A y B se sobreponen.");
      Rectangulo S = rectanguloS(A, B);
      System.out.printf("Area de sobreposicion = %.2f\n", S.calculoArea());
    } else if (Verificador.esJunto(A, B)) {
      System.out.println("Rectangulos A y B se juntan");
    } else if (Verificador.esDisjunto(A, B)) {
      System.out.println("Rectangulos A y B son disjuntos");
    }

    sc.close();
  }

  public static Rectangulo rectanguloS(Rectangulo r1, Rectangulo r2) {

    double x_izq, y_abj, x_der, y_arb;

    if (r1.getEsquina1().getX() > r2.getEsquina1().getX()) {
      x_izq = r1.getEsquina1().getX();
    } else {
      x_izq = r2.getEsquina1().getX();
    }

    if (r1.getEsquina1().getY() > r2.getEsquina1().getY()) {
      y_abj = r1.getEsquina1().getY();
    } else {
      y_abj = r2.getEsquina1().getY();
    }
```

```
    if (r1.getEsquina2().getX() < r2.getEsquina2().getX()) {
      x_der = r1.getEsquina2().getX();
    } else {
      x_der = r2.getEsquina2().getX();
    }

    if (r1.getEsquina2().getY() < r2.getEsquina2().getY()) {
      y_arb = r1.getEsquina2().getY();
    } else {
      y_arb = r2.getEsquina2().getY();
    }

    return new Rectangulo(new Coordenada(x_izq, y_abj), new Coordenada(x_der, y_arb));
  }
}
```

4 Ejercicios

4.1 Ejercicio N°1: Contenedor de Rectángulos

Esta clase guarda un arreglo de rectángulos junto con sus áreas y distancias diagonales precalculadas. El atributo `numRec` es estático, lo que significa que es compartido entre todas las instancias. El método `toString` muestra todo en una tabla formateada con los datos de cada rectángulo.

```java
public class ContainerRect {
    private Rectangulo[] rectangulos;
    private double[] distancias;
    private double[] areas;
    private int n;
    private static int numRec = 0;

    public ContainerRect(int n) {
        this.n = n;
        this.rectangulos = new Rectangulo[n];
        this.distancias = new double[n];
        this.areas = new double[n];
    }

    public void addRectangulo(Rectangulo r) {
        if (numRec < n) {
            rectangulos[numRec] = r;
            distancias[numRec] = r.getEsquina1().distancia(r.getEsquina2());
            areas[numRec] = r.calculoArea();
            numRec++;
        } else {
            System.out.println("Ya no es posible guardar el rectangulo");
        }
    }

    public String toString() {
        String tabla = String.format("%-12s %-30s %-12s %-10s\n", "Rectangulo", "Coordenadas", "Distancia", "Area");
        for (int i = 0; i < numRec; i++) {
            tabla += String.format("%-12d %-30s %-12.3f %-10.2f\n",
                    (i + 1), rectangulos[i].toString(), distancias[i], areas[i]);
        }
        return tabla;
    }
}
```

4.2 Ejercicio N°2: Análisis de una Zona Minera

El main pide la ruta del archivo y el tamaño `k` al usuario, carga la matriz con `LectorArchivo` y llama a `mostrarMejorSubregion` que imprime la posición, las zonas y el valor total de la mejor subregión encontrada.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("EJERCICIO 2 MINERALES - PUREZA");
        System.out.print("Ingrese la ruta del archivo de datos (.txt): ");
        String ruta = sc.nextLine();

        MatrizMinera matriz = LectorArchivo.leerArchivo(ruta);

        if (matriz == null) {
```

```
            System.out.println("No se pudo cargar la matriz. Terminando programa.");
            sc.close();
            return;
        }

        System.out.print("Ingrese el tamano k de la subregion cuadrada (k x k): ");
        int k = sc.nextInt();


        matriz.mostrarMejorSubregion(k);

        sc.close();
    }
}
```

Guarda todas las zonas en una matriz bidimensional. El método `encontrarMejorSubregion` recorre todas las posiciones posibles y calcula el valor total de cada subregión de tamaño $k \times k$, quedándose con la de mayor valor. `getMineralPredominante` cuenta manualmente cuántas veces aparece cada mineral usando arreglos.

```
public class MatrizMinera {
    private Zona[][] matriz;
    private int filas;
    private int columnas;

    public MatrizMinera(int filas, int columnas) {
        this.filas = filas;
        this.columnas = columnas;
        this.matriz = new Zona[filas][columnas];
    }

    public void setZona(int fila, int columna, Zona zona) {
        matriz[fila][columna] = zona;
    }

    public Zona getZona(int fila, int columna) {
        return matriz[fila][columna];
    }

    public int getFilas() {
        return filas;
    }

    public int getColumnas() {
        return columnas;
    }

    public double calcularValorSubregion(int filaInicio, int colInicio, int k) {
        double total = 0;
        for (int i = filaInicio; i < filaInicio + k; i++) {
            for (int j = colInicio; j < colInicio + k; j++) {
                total += matriz[i][j].getValorEconomico();
            }
        }
        return total;
    }

    public String getMineralPredominante(int filaInicio, int colInicio, int k) {
        String[] minerales = new String[k * k];
        int[] conteos = new int[k * k];
        int totalMinerales = 0;

        for (int i = filaInicio; i < filaInicio + k; i++) {
            for (int j = colInicio; j < colInicio + k; j++) {
                String tipo = matriz[i][j].getTipoMineral();
                boolean encontrado = false;
                for (int m = 0; m < totalMinerales; m++) {
                    if (minerales[m].equals(tipo)) {
                        conteos[m]++;
                        encontrado = true;
```

```
                    break;
                }
            }
            if (!encontrado) {
                minerales[totalMinerales] = tipo;
                conteos[totalMinerales] = 1;
                totalMinerales++;
            }
        }
    }

    String predominante = minerales[0];
    int maxConteo = conteos[0];
    for (int m = 1; m < totalMinerales; m++) {
        if (conteos[m] > maxConteo) {
            maxConteo = conteos[m];
            predominante = minerales[m];
        }
    }
    return predominante;
}

public int[] encontrarMejorSubregion(int k) {
    if (k > filas || k > columnas) {
        System.out.println("Error: k es mayor que las dimensiones de la matriz.");
        return null;
    }

    double maxValor = -1;
    int mejorFila = 0;
    int mejorCol = 0;

    for (int i = 0; i <= filas - k; i++) {
        for (int j = 0; j <= columnas - k; j++) {
            double valor = calcularValorSubregion(i, j, k);
            if (valor > maxValor) {
                maxValor = valor;
                mejorFila = i;
                mejorCol = j;
            }
        }
    }

    return new int[]{mejorFila, mejorCol};
}

public void mostrarMejorSubregion(int k) {
    int[] posicion = encontrarMejorSubregion(k);
    if (posicion == null) return;

    int fila = posicion[0];
    int col = posicion[1];
    double valorTotal = calcularValorSubregion(fila, col, k);
    String mineralPred = getMineralPredominante(fila, col, k);

    System.out.println("Región más valiosa encontrada:\n");

    System.out.println("Posición inicial: (" + fila + ", " + col + ")");
    System.out.println("Tamaño de la región: " + k + " x " + k + "\n");

    System.out.println("Zonas analizadas:");
    for (int i = fila; i < fila + k; i++) {
        for (int j = col; j < col + k; j++) {
            Zona z = matriz[i][j];
            System.out.println("[ " + z.getTipoMineral() +
                    ", cantidad: " + z.getCantidad() +
                    ", pureza: " + z.getPureza() + " ]");
        }
    }

    System.out.println("\nValor total estimado: " + String.format("%.2f", valorTotal));

    System.out.println("\nMineral predominante en la región: " + mineralPred);
```

```
    }
}
```

Representa una celda de la matriz minera con un tipo de mineral, una cantidad y un nivel de pureza. El método `getValorEconomico` multiplica cantidad por pureza para obtener el valor de esa zona.

```
public class Zona {
    private String tipoMineral;
    private int cantidad;
    private double pureza;

    public Zona(String tipoMineral, int cantidad, double pureza) {
        this.tipoMineral = tipoMineral;
        this.cantidad = cantidad;
        this.pureza = pureza;
    }

    public double getValorEconomico() {
        return cantidad * pureza;
    }

    public String getTipoMineral() {
        return tipoMineral;
    }

    public int getCantidad() {
        return cantidad;
    }

    public double getPureza() {
        return pureza;
    }

    @Override
    public String toString() {
        return tipoMineral + " cantidad: " + cantidad + "  pureza: " + pureza
                + "  Valor: " + String.format("%.2f", getValorEconomico());
    }
}
```

Lee el archivo `.txt` línea por línea: la primera línea tiene las dimensiones de la matriz y las siguientes tienen los datos de cada zona separados por espacios. Usa `trim().split(" ")` para limpiar los datos y los convierte al tipo correcto antes de crear cada objeto `Zona`.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LectorArchivo {

    public static MatrizMinera leerArchivo(String ruta) {
        MatrizMinera matrizMinera = null;

        try {
            BufferedReader br = new BufferedReader(new FileReader(ruta));

            String primeraLinea = br.readLine();
            String[] dimensiones = primeraLinea.trim().split(" ");
            int filas = Integer.parseInt(dimensiones[0]);
            int columnas = Integer.parseInt(dimensiones[1]);

            matrizMinera = new MatrizMinera(filas, columnas);

            for (int i = 0; i < filas; i++) {
```

```java
                for (int j = 0; j < columnas; j++) {
                    String linea = br.readLine();
                    if (linea == null) {
                        System.out.println("Error: el archivo tiene menos datos de los esperados.");
                        br.close();
                        return null;
                    }
                    String[] partes = linea.trim().split(" ");
                    String tipoMineral = partes[0];
                    int cantidad = Integer.parseInt(partes[1]);
                    double pureza = Double.parseDouble(partes[2]);

                    Zona zona = new Zona(tipoMineral, cantidad, pureza);
                    matrizMinera.setZona(i, j, zona);
                }
            }

            br.close();
            System.out.println("Archivo leido correctamente: " + filas + "x" + columnas + " zonas cargadas.");

        } catch (IOException e) {
            System.out.println("Error al leer el archivo: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error en el formato del archivo: " + e.getMessage());
        }

        return matrizMinera;
    }
}
```

```
3 3
Oro 10 0.9
Plata 8 0.85
Cobre 12 0.7
Oro 6 0.92
Oro 4 0.88
Plata 7 0.8
Cobre 9 0.75
Oro 5 0.9
Plata 6 0.82
```

5 Dificultades Encontradas

Al momento de implementar la clase `Verificador`, me costó bastante entender cómo detectar correctamente cuándo dos rectángulos se tocan sin sobreponerse. La diferencia entre "junto" y "sobreposición" no era tan obvia al principio, y tuve que pensar bien en qué condiciones comparten un borde sin tener área en común.

En el Ejercicio 2, otra dificultad fue leer correctamente el archivo `.txt` y separar bien cada dato por línea. Al principio no tenía claro cómo manejar los espacios entre los valores, y el programa fallaba al intentar convertir los datos a `int` o `double`.

6 Soluciones Implementadas

Para el problema con `Verificador`, lo resolví pensando en los casos por separado: primero verifico si hay sobreposición, y si no la hay, reviso si comparten exactamente un borde con rango válido en el eje contrario. Dividir el problema en tres métodos (`esSobrePos`, `esJunto`, `esDisjunto`) me ayudó a ordenar mejor la lógica.

Para la lectura del archivo, usé `trim().split(" ")` para limpiar espacios y separar cada línea en sus partes. También agregué mensajes de error con `try-catch` para identificar rápido si el archivo tenía un formato incorrecto o si la ruta estaba mal escrita.

7 Conclusiones

- Los arreglos de objetos son una estructura de datos lineal que permite almacenar colecciones de elementos del mismo tipo, y su recorrido mediante ciclos `for` es el algoritmo más básico para procesar todos los elementos almacenados en ellas.
- El encapsulamiento no es solo una buena práctica de programación, sino también una forma de proteger la integridad de la estructura de datos, ya que al declarar los atributos como `private` se evita que los datos se modifiquen de forma incorrecta desde fuera de la clase.
- La separación entre métodos de instancia y métodos estáticos permite distinguir entre operaciones que dependen del estado de un objeto específico y algoritmos generales que solo necesitan los datos que reciben como parámetros, como los métodos de verificación en la clase `Verificador`.
- La composición entre clases, como el uso de objetos `Coordenada` dentro de `Rectangulo` y objetos `Rectangulo` dentro de `ContainerRect`, es la base para construir estructuras de datos más complejas, donde una estructura contiene a otras como parte de su diseño.

8 Bibliografía

Guevara, K., Flores, O., Galarza, M., Reinoso, G., & Manrique, L. (2026). *Laboratorio 01: POO – Utilizando clases y arreglos de objetos.* Algoritmos y Estructura de Datos, Universidad Católica de Santa María, Escuela Profesional de Ingeniería de Sistemas.

9 Anexos

ANEXO A:

Repositorio que almacena los códigos de los ejercicios y actividades realizados durante el Laboratorio N°1
`https://github.com/lucaespinozaro/Lab01_AED`